

Lab02_ME315

Pedro Tilhe 254672

```
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

```
filter, lag
```

Os seguintes objetos são mascarados por 'package:base':

```
intersect, setdiff, setequal, union
```

```
library(readr)  
library(tibble)  
library(ggplot2)  
library(ggcal)
```

Laboratório 2 — Processamento de Bases de Dados em Lote

Questão 1

Enunciado: Quais são as estatísticas suficientes para a determinação do percentual de voos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

As estatísticas suficientes são o número total de voos válidos e o número de voos cuja chegada apresentou atraso superior a 10 minutos, considerando cada combinação de companhia aérea e data.

O percentual de atrasos pode ser calculado por:

$$Perc = \frac{\text{Número de voos atrasados}}{\text{Número total de voos}}$$

Questão 2

Enunciado: Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas:

- Filtre o conjunto de dados de forma que contenha apenas observações das companhias aéreas AA, DL, UA e US;
- Remova observações que tenham valores faltantes em campos de interesse;
- Agrupe o conjunto de dados resultante de acordo com dia, mês e companhia aérea;
- Para cada grupo, determine as estatísticas suficientes apontadas na questão anterior e retorne-as como um objeto da classe `tibble`;
- A função deve receber apenas os argumentos `input` e `pos`.

```
getStats <- function(input, pos) {

  input %>%
    filter(
      AIRLINE %in% c("AA", "DL", "UA", "US"),
      !is.na(AIRLINE),
      !is.na(MONTH),
      !is.na(DAY),
      !is.na(ARRIVAL_DELAY)
    ) %>%
    group_by(AIRLINE, MONTH, DAY) %>%
    summarise(
      Total = n(),
      Atrasados = sum(ARRIVAL_DELAY > 10),
      .groups = "drop"
    ) %>%
    as_tibble()
}
```

Questão 3

Enunciado: Utilize alguma função `readr::read*_chunked` para importar o arquivo `flights.csv.zip`.

Configure o tamanho de cada lote para 100 mil registros, utilize a função `getStats` como `callback` e carregue apenas as colunas necessárias para a resolução do exercício.

```
callback <- DataFrameCallback$new(getStats)

statsChunks <- read_csv_chunked(
  "flights.csv.zip",
  callback = callback,
  chunk_size = 100000,
  col_types = cols_only(
    MONTH = col_double(),
    DAY = col_double(),
    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)
```

Questão 4

Enunciado: Crie uma função chamada `computeStats` que combine as estatísticas suficientes obtidas nos diferentes lotes e produza a métrica final de interesse.

A função deve retornar um `tibble` contendo apenas:

- `Cia`: sigla da companhia aérea;
- `Data`: data no formato AAAA-MM-DD;
- `Perc`: percentual de atraso da companhia naquela data, representado por um número real no intervalo $[0,1]$.

```
computeStats <- function(stats) {

  stats %>%
    group_by(AIRLINE, MONTH, DAY) %>%
    summarise(
      Total = sum(Total),
      Atrasados = sum(Atrasados),
      .groups = "drop"
    ) %>%
    mutate(
      Cia = AIRLINE,
      Data = as.Date(
        paste(2015, MONTH, DAY, sep = "-")
      ),
      Perc = Atrasados / Total
    ) %>%
}
```

```

    select(Cia, Data, Perc) %>%
      as_tibble()
  }

stats <- computeStats(statsChunks)

```

Resultado consolidado

```
stats
```

```

# A tibble: 1,276 x 3
   Cia   Data      Perc
   <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 AA    2015-01-02 0.430
3 AA    2015-01-03 0.538
4 AA    2015-01-04 0.602
5 AA    2015-01-05 0.491
6 AA    2015-01-06 0.366
7 AA    2015-01-07 0.355
8 AA    2015-01-08 0.307
9 AA    2015-01-09 0.342
10 AA   2015-01-10 0.280
# i 1,266 more rows

```

Questão 5

Enunciado: Produza um mapa de calor em formato de calendário para cada companhia aérea.

Instale e carregue os pacotes `ggcal` e `ggplot2`. Defina uma paleta de cores em gradiente utilizando `scale_fill_gradient`, com cor inicial `#4575b4` e cor final `#d73027`.

```

pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)

```

Questão 5.c — Função baseCalendario

Enunciado: Crie uma função chamada `baseCalendario` que receba dois argumentos: `stats`, contendo os resultados calculados, e `cia`, representando a companhia aérea de interesse.

A função deve selecionar os dados da companhia indicada e gerar a base do calendário utilizando `ggcal(x, y)`.

```
baseCalendario <- function(stats, cia) {  
  
  dadosCia <- stats %>%  
    filter(Cia == cia) %>%  
    select(Data, Perc)  
  
  ggcal(dadosCia$Data, dadosCia$Perc)  
}
```

Questão 5.d — Calendários das companhias

Enunciado: Execute a função `baseCalendario` para cada uma das companhias aéreas e armazene os resultados nas variáveis `cAA`, `cDL`, `cUA` e `cUS`.

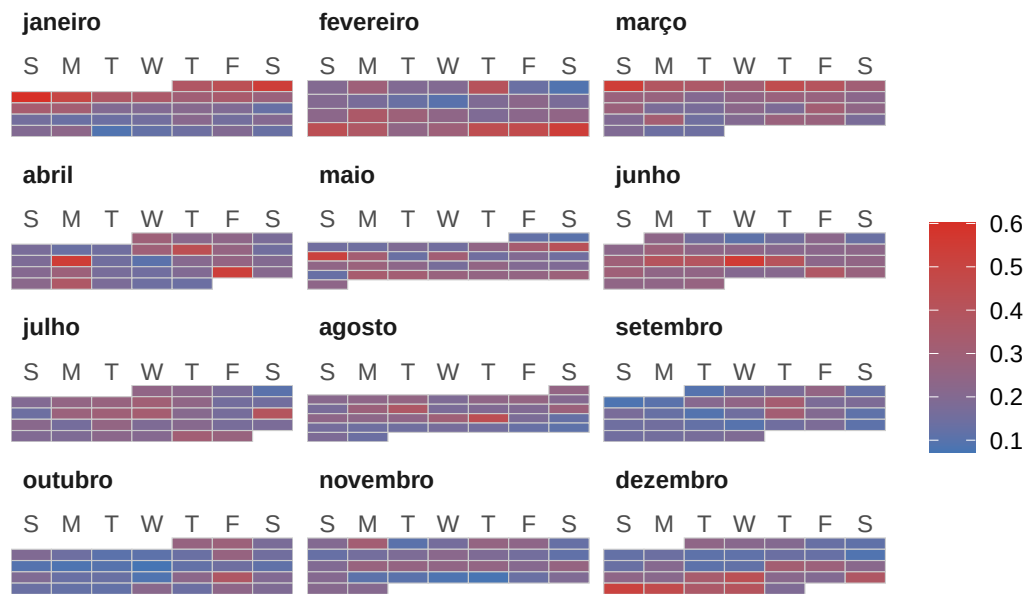
```
cAA <- baseCalendario(stats, "AA")  
cDL <- baseCalendario(stats, "DL")  
cUA <- baseCalendario(stats, "UA")  
cUS <- baseCalendario(stats, "US")
```

Questão 5.e — American Airlines

Enunciado: Apresente o mapa de calor referente à companhia AA utilizando a paleta definida anteriormente e adicione um título ao gráfico.

```
cAA + pal + ggtitle("American Airlines - AA")
```

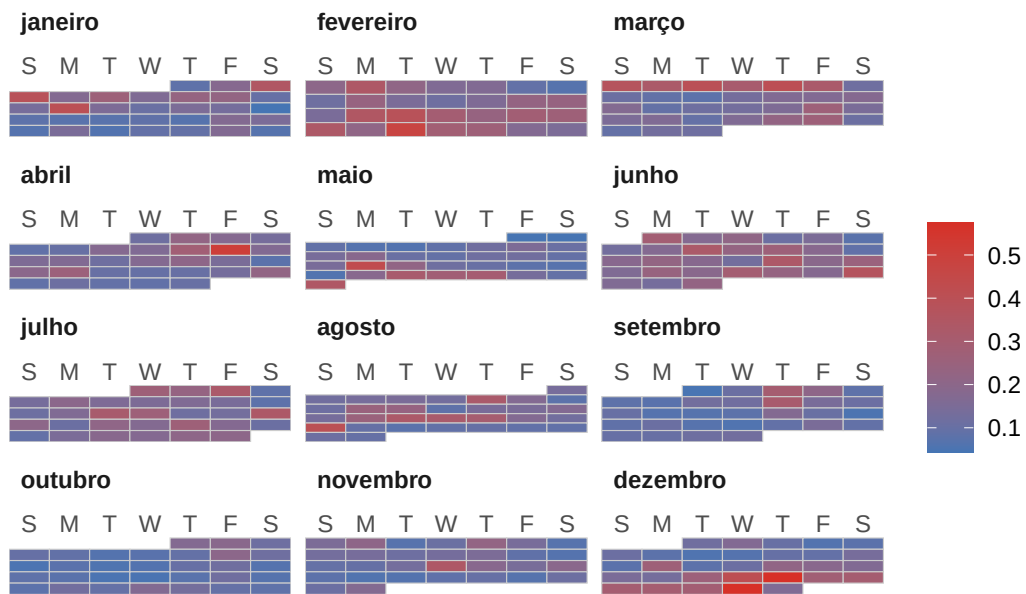
American Airlines – AA



Delta Air Lines

```
cDL + pal + ggtitle("Delta Air Lines - DL")
```

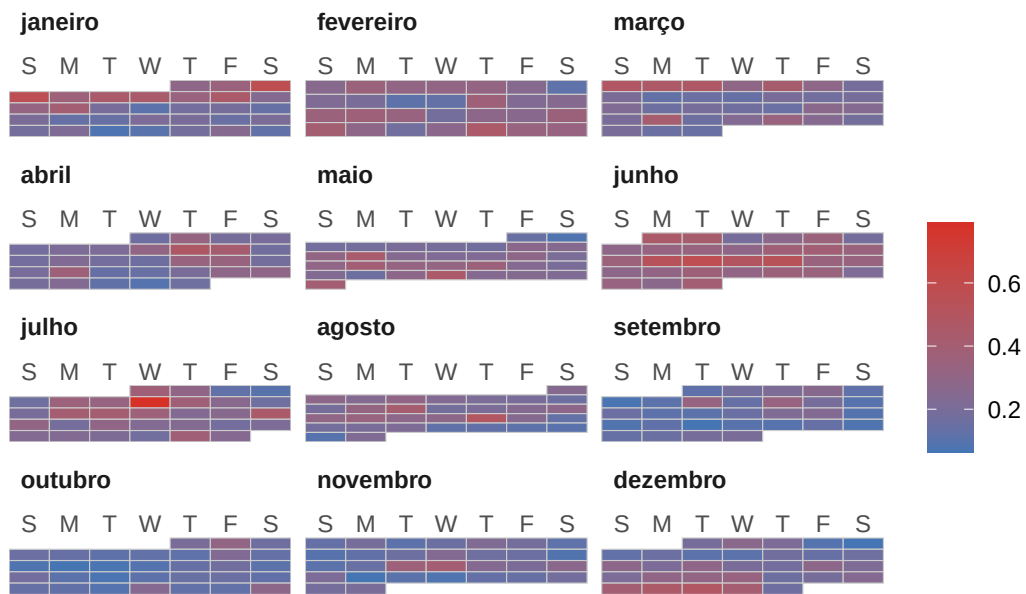
Delta Air Lines – DL



United Airlines

```
cUA + pal + ggtitle("United Airlines - UA")
```

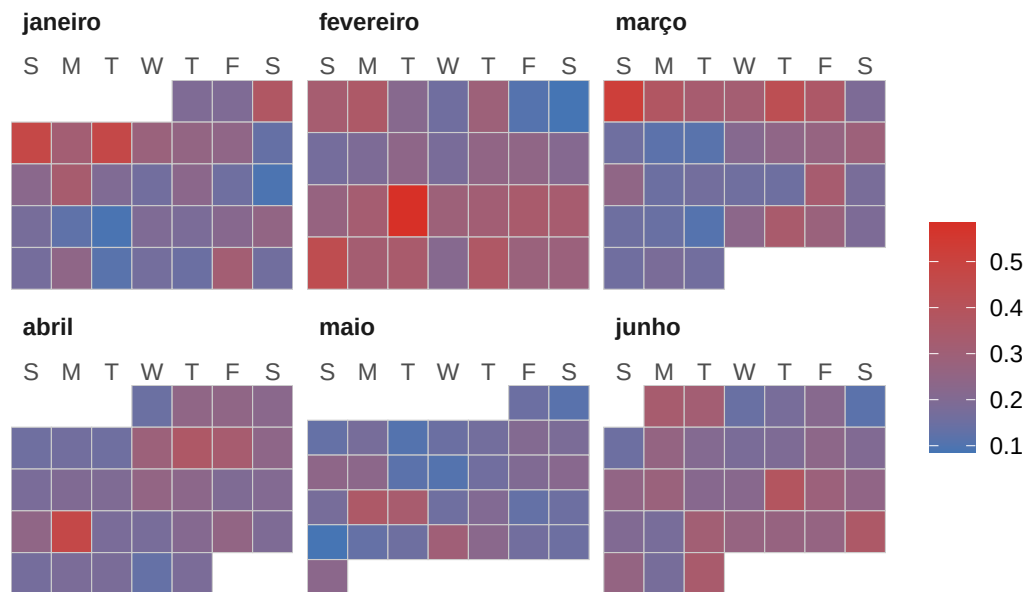
United Airlines – UA



US Airways

```
cUS + pal + ggtitle("US Airways - US")
```

US Airways – US



Horário de compilação

Enunciado: Incluir um chunk R ao final do documento para registrar o horário em que o arquivo foi compilado, configurado para o horário do Brasil.

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-24 21:02:13 -03"
```